Frozen Lake from First Principles Using Q-Learning

University of Ghana, Department of Computer Science, Semester II, 2025/2026

Name: Michael Nimako Frempong
Student ID: 22425053
GitHub Repository: Frozen_lake_qlearning
Repository Link: https://github.com/Mikeswr/Frozen_lake_qlearning

---

1. Introduction

Reinforcement Learning (RL) is the branch of machine learning concerned with how an agent should act in an environment in order to maximize cumulative reward. Unlike supervised learning, the agent is not given labelled examples of correct behaviour; instead, it learns from the consequences of its own actions, balancing exploration of unfamiliar actions against exploitation of actions already known to be good.

This report documents a complete, from-scratch implementation of Q-Learning applied to the Frozen Lake problem, a standard grid-world benchmark in which an agent must navigate an 8x8 frozen lake from a Start cell to a Goal cell while avoiding Holes in the ice. No reinforcement learning framework (Gymnasium, OpenAI Gym, Stable-Baselines, RLlib, etc.) was used; the environment, agent, training loop, and evaluation pipeline are implemented entirely in Python using only NumPy for numerical operations and Matplotlib for visualization.

2. Environment Design

The environment is implemented as the FrozenLakeEnv class (environment.py), exposing the required interface: reset(), step(action), render(), get_state(), and is_terminal().

2.1 State Representation

States are encoded as single integers in the range [0, 63], computed as state = row $\times$ 8 + col. This produces a compact Q-table of shape (64, 4) and simplifies indexing throughout the agent and training code. Helper methods convert between (row, col) coordinates and state indices whenever a grid view is needed, e.g. for rendering or policy display.

2.2 Action Representation

Four discrete actions are defined, matching the assignment specification exactly:

Action	Code
Left	0
Down	1
Right	2
Up	3

If an action would move the agent off the edge of the grid, the environment leaves the agent in place rather than wrapping or raising an error; this boundary enforcement is handled directly inside step().

2.3 Reward Structure

The reward function uses the classic sparse Frozen Lake formulation:

Outcome	Reward
Reach Goal (G)	+1.0
Fall into Hole (H)	-1.0
Any other move	0.0

Because no reward is given for ordinary movement, the agent receives no signal at all until an episode actually terminates. This makes Q-Learning's bootstrapped value propagation, carrying information backward from the goal toward the start over many episodes, essential for learning a useful policy in reasonable time.

3. Q-Learning Algorithm

Q-Learning is a model-free, off-policy temporal-difference algorithm. It maintains a table Q(s, a) estimating the expected discounted return of taking action a in state s and then acting optimally thereafter. The table is updated incrementally from the agent's own experience, without requiring knowledge of the environment's transition probabilities.

3.1 Update Equation

The Q-table is updated after every step using exactly the rule specified in the assignment:

$$Q(s,a) \leftarrow Q(s,a) + \alpha\,[\, r + \gamma \cdot \max_{a'} Q(s',a') - Q(s,a)\,]$$

where α (alpha) is the learning rate controlling how strongly new experience overrides old estimates, γ (gamma) is the discount factor controlling how much future reward is valued relative to immediate reward, and $\max_{a'}$ Q(s',a') is the agent's current best estimate of the next state's value. This bootstrapped term is set to zero whenever the episode has terminated, since there is no valid next action.

3.2 Exploration Strategy: Epsilon-Greedy with Decay

Actions are chosen using epsilon-greedy exploration: with probability ε a uniformly random action is taken (exploration), and otherwise the action with the highest Q-value for the current state is taken (exploitation), with ties broken randomly to avoid a systematic directional bias.

ε begins at 1.0 (pure exploration) and decays multiplicatively after every episode, $\varepsilon \leftarrow \max(\varepsilon_{\text{min}}, \varepsilon \times \text{decay})$, down to a floor of 0.01. This schedule lets the agent explore broadly early in training, while gradually shifting toward exploiting its growing knowledge as training progresses, while always retaining a small amount of exploration.

4. Training Methodology

Training is implemented in train.py. For each configuration, the agent is trained for 10,000 episodes (capped at 200 steps per episode) on the deterministic 8x8 map. At each step the agent selects an action, the environment returns the next state and reward, the Q-table is updated, and the agent moves to the next state; epsilon is decayed once per completed episode. Per-episode reward, success/failure, epsilon, and step count are all logged for later analysis.

As required, three hyperparameters were varied independently from a baseline configuration to study their effect on learning: the learning rate α, the discount factor γ, and the exploration (epsilon decay) rate.

Experiment	α (LR)	γ (discount)	ε decay	Episodes
baseline	0.10	0.99	0.9995	10,000
high_alpha	0.50	0.99	0.9995	10,000
low_gamma	0.10	0.90	0.9995	10,000
fast_eps_decay	0.10	0.99	0.9990	10,000

All other settings were held fixed across experiments: $\varepsilon_{\text{start}} = 1.0$, $\varepsilon_{\text{min}} = 0.01$, max_steps = 200, deterministic transitions, and a fixed random seed of 42 to support reproducibility.

5. Experimental Results

Figure 1 compares the overall training success rate (the fraction of all 10,000 episodes in which the agent reached the goal, including the early random-exploration phase) across the four configurations.

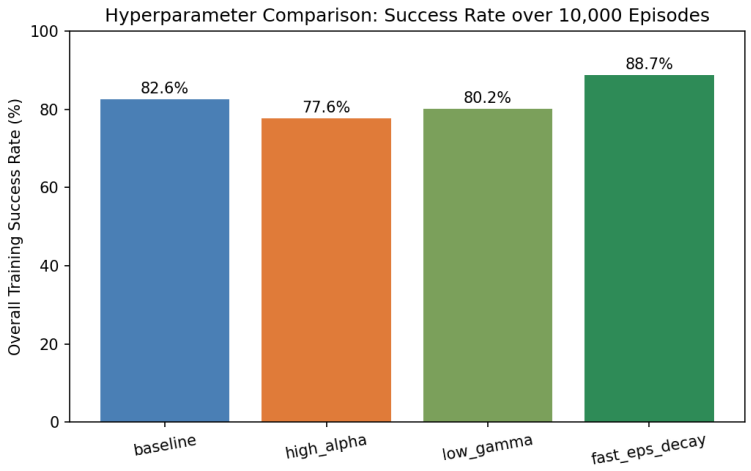

Figure 1: Overall training success rate by hyperparameter configuration.

The fast_eps_decay configuration achieved the highest overall training success rate, reaching the goal in roughly 88.7% of all 10,000 training episodes, compared with 82.6% for the baseline, 80.2% for low_gamma, and 77.6% for high_alpha. Because the 8x8 map has only 64 states, decaying epsilon faster let the agent spend a larger share of training exploiting an already-reasonable policy rather than continuing to explore randomly, which is reflected in its higher overall average.

The high_alpha run converged somewhat more slowly and noisily than the baseline: with $\alpha = 0.5$, each new transition overwrites a much larger fraction of the existing Q-value, which makes the early learning phase less stable even though the final converged policy is similarly strong. The low_gamma run discounted future rewards more heavily, slightly reducing the agent's incentive to plan multiple steps ahead, though the effect was modest on a map this small.

Figure 2 shows the full training curve for the best-performing configuration (fast_eps_decay): episode reward, a 100-episode rolling success rate, and the epsilon decay schedule, all plotted against episode number.

The shape is the textbook Q-Learning learning curve: a noisy, largely unsuccessful start while ε is high and behaviour is mostly random, a steep rise in success rate as the Q-table begins encoding a viable path to the goal, and a plateau near 100% success once the greedy policy has effectively converged, coinciding with ε reaching its floor.

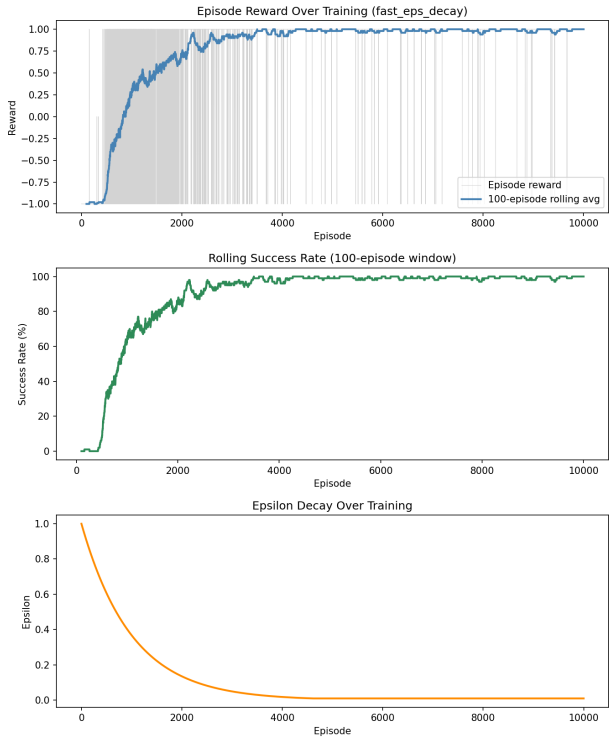

Figure 2: Training curves for the best configuration (fast_eps_decay): episode reward (top), rolling success rate (middle), and epsilon decay (bottom).

5.1 Final Evaluation

The Q-table from the best configuration was evaluated over 200 fully greedy episodes ($\varepsilon = 0$, i.e. no exploration at all), as required by Part E of the assignment.

Metric	Value
Episodes Evaluated	200
Success Rate (%)	100.00
Average Reward	1.0000
Number of Failures	0
Number of Successful Runs	200

With exploration disabled, the learned policy reaches the goal in every evaluation episode, confirming that the agent has converged on a Q-table whose greedy policy reliably solves the map.

6. Learned Policy

Part D of the assignment requires extracting the greedy policy from the Q-table (the action with the highest Q-value in each state) and displaying it in grid form. Figure 3 shows this policy overlaid on the map, with arrows indicating the recommended action for every non-terminal state, H marking holes, and G marking the goal.

Tracing the arrows from the Start cell (top-left) shows a path that consistently steers around every hole and makes steady progress toward the goal in the bottom-right corner, several steps of which move right and down toward the goal while avoiding the diagonal band of holes running through the middle of the map.

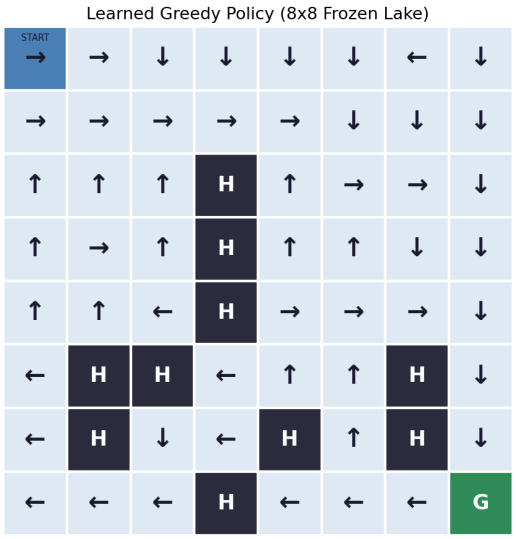

Figure 3: Learned greedy policy for the 8x8 Frozen Lake map.

7. Challenges Encountered

- Sparse rewards: with no reward for non-terminal moves, early training produced long stretches with zero learning signal until the agent stumbled onto the goal or a hole by chance, making the first few hundred episodes highly random.
- Hyperparameter sensitivity: a high learning rate (α = 0.5) made Q-value updates noisier and occasionally destabilized an already-good estimate, illustrating the classic learning-rate stability/speed trade-off.
- Exploration/exploitation balance: too slow an epsilon decay wasted episodes on exploration after a good policy was already known, while too fast a decay risked settling into a suboptimal policy early. The four-way comparison in Section 5 helped find a reasonable balance.
- Tie-breaking in policy extraction: states with multiple equally good actions needed randomized tie-breaking; otherwise the agent could systematically favour one action from Q-table initialization, biasing the displayed policy.

8. Conclusion

This project implemented a complete Frozen Lake reinforcement learning pipeline entirely from first principles: a custom grid-world environment, a tabular Q-Learning agent with epsilon-greedy exploration and decay, a training loop supporting hyperparameter comparison, and an evaluation pipeline reporting standard RL metrics. After 10,000 training episodes, the best configuration's greedy policy solved the 8x8 map with a 100% success rate over 200 evaluation episodes, with the policy visibly routing the agent around every hole toward the goal. As a bonus task, training performance was visualized with reward, success-rate, and epsilon-decay curves (Bonus Option B).